

基于Harness工程化框架的CDD平台化2.0设计探索

日期	版本	修订内容
2026.7.2	1.0	初始版本
2026.7.7	2.0	完善版本
2026.7.20	3.0	预审建议完善版本

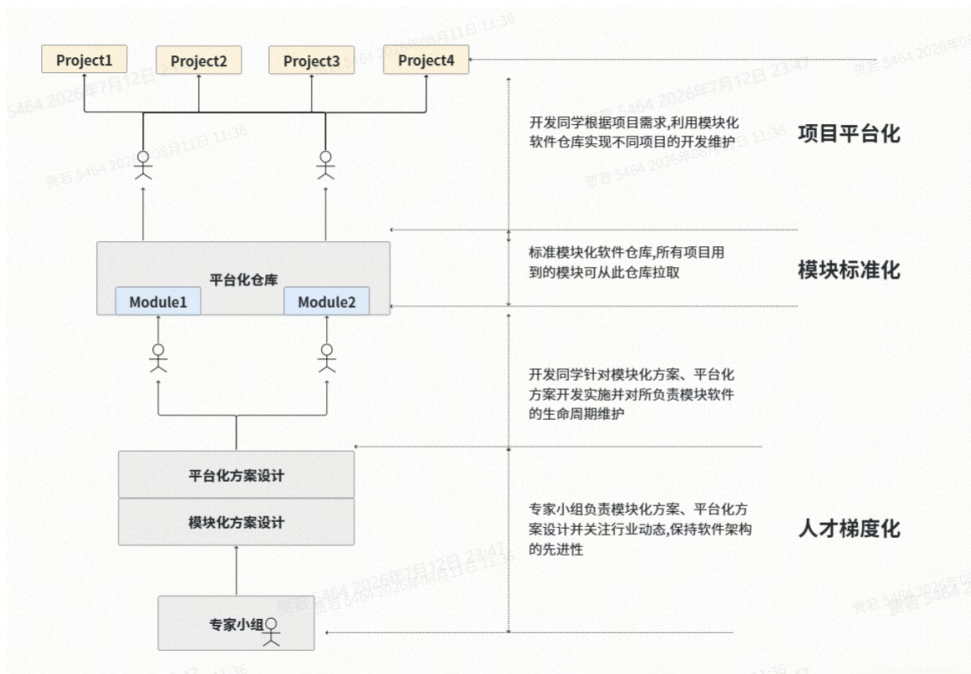
缩略词

缩略词	英文全名	中文解释
CDD	Complex Device Driver	复杂设备驱动
ARXML	AUTOSAR XML	AUTOSAR XML配置文件
LLM	Large Language Model	大语言模型
Harness Engineering	Harness Engineering	驾驭工程
MCP	Model Context Protocol	模型上下文协议
Agent	AI Agent	Ai智能体
Terminal Bench	Terminal Bench	斯坦福大学与Laude Institute联合打造的一个AI Agent能力评测基准，目前已成为评估智能体（Agent）真实工程能力的核心标尺

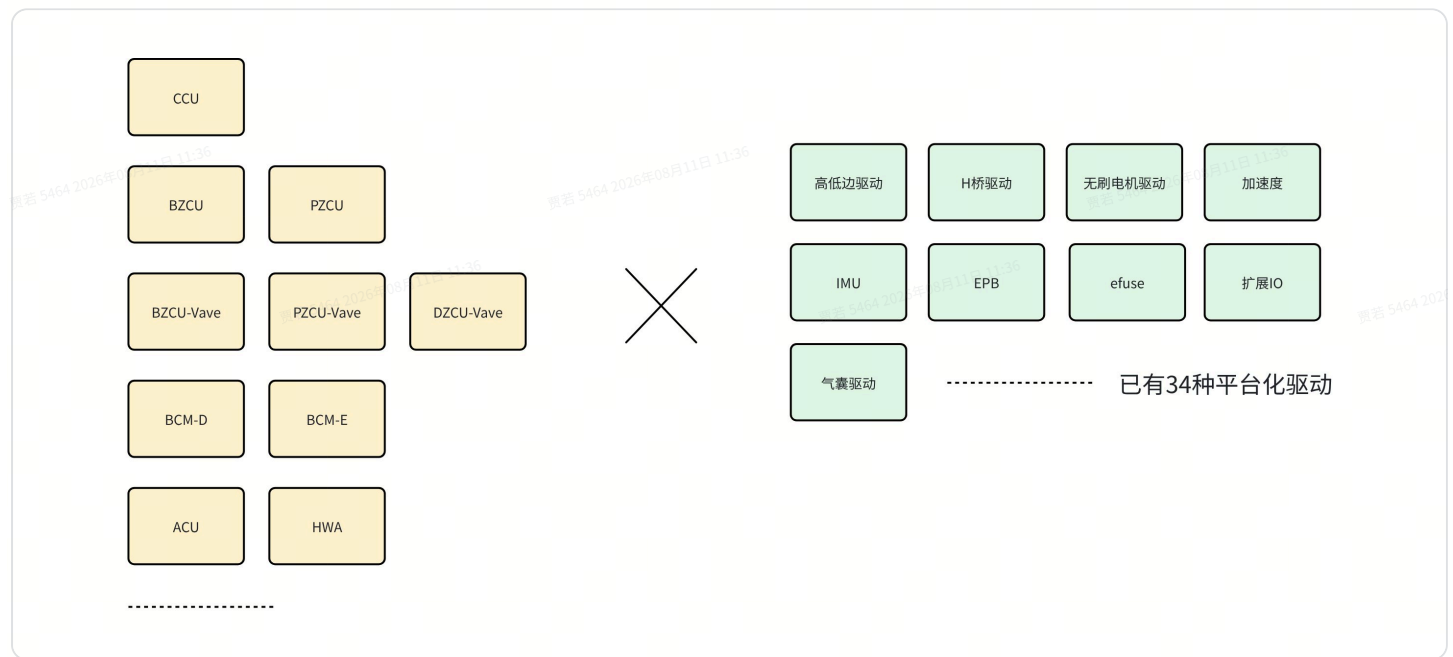
1. 当前描述

1.1 CDD平台化1.0痛点

CDD平台化1.0已完成驱动逻辑与配置代码分离，建立了模块专家制定标准、软件开发沉淀标准化模块、平台化仓库统一管理、项目侧按需配置复用的基本模式。这意味着团队已经初步解决了驱动能力难复用、模块开发分散的问题，形成了CDD驱动平台化的基础能力。



但从项目交付实际情况看，当前平台化能力的主要瓶颈，已不再是驱动逻辑本身，而是配置代码仍主要依赖人工手写。虽然底层驱动已完成平台化沉淀，但在不同项目落地时，工程师仍需要根据芯片资源、引脚分配、业务差异手工编写或修改配置代码。这一环节直接决定平台化能力能否高效、稳定地释放到具体项目中。



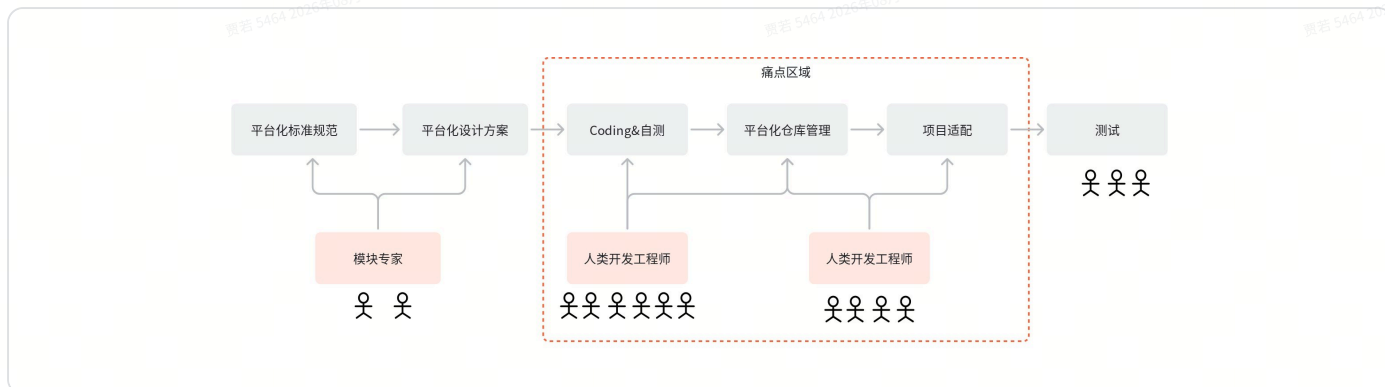
因此，CDD平台化1.0当前面临的核心问题，不是“有没有平台化”，而是平台化后的配置交付方式仍然停留在人工模式。这使得平台化成果在跨项目复制时，仍然受到人工效率、个人经验和执行一致性的制约。

1. 平台化设计

- 模块专家小组负责制定模块化及平台化标准规范，并输出方案设计给到软件开发同学。
- 软件开发同学基于标准规范和设计方案，对所负责模块进行开发和生命周期维护。
- 开发完成的标准化模块统一提交平台化仓库进行管理，并提供给各项目使用。

- 项目开发同学根据项目需求实施定制化配置，实现不同项目的开发维护。

2. 开发痛点



- 风格难以统一：**人类工程师编码风格各异，即使有编码规范，也难以在代码风格、缩进、命名和注释习惯上做到完全统一，导致配置代码可读性差、维护成本高。
- 手写易出错：**人工编写配置代码容错率低，容易因拼写错误、符号缺失、参数遗漏或引脚冲突导致编译失败甚至运行异常。
- 手写效率低：**随着芯片外设数量和业务复杂度增加，人工查阅手册并手写结构体、宏定义和初始化配置的工作量持续上升，交付效率受到明显影响。
- 重编码轻设计：**在交付周期紧、节奏快的情况下，工程师往往边思考边Coding，功能跑通后即提交入库，详细设计和自测环节容易弱化，导致问题后移和质量风险累积。

🏆 综上，CDD平台化1.0虽然解决了驱动逻辑复用和模块沉淀问题，但配置代码仍依赖人工手写，已经成为当前影响平台化能力继续释放的主要瓶颈。后续若要进一步提升平台化交付效率和一致性质量，就必须围绕配置代码生成方式进行升级。

1.2 AI 开发驱动软件的痛点

III AI 写嵌入式代码的痛点不是"代码写得不好"，而是：

1. 嵌入式代码训练数据不足
2. 上下文多源
3. 安全合规无法保证
4. 实时性约束不可见
5. 调试反馈回路断裂
6. 资源约束感知缺失

1. 训练数据不足

LLM的能力上限受训练数据影响显著。GitHub上Python、JavaScript、Java等代码海量存在，但嵌入式C、AUTOSAR代码和芯片驱动相关代码稀缺，导致模型在该领域容易出现“表面合理、细节错误”的生成结果。

AI能写出一个漂亮的React组件，但若让其编写一段TriCore DMA双缓冲配置，很可能会编造寄存器地址或混用不同芯片系列API。

2. 上下文多源

这是最根本的痛点。嵌入式开发的"上下文"不只是代码仓库，而是：

层次	AI 的理解	实际需要
芯片架构	大概知道 Cortex-M3/M4 的区别	需要知道具体型号的内存映射、中断向量表、外设互联矩阵
寄存器	能背几个通用寄存器名	需要 2000+ 页 datasheet 中的精确位域定义
时序	概念欠缺	ISR 延迟几微秒就可能导致通信丢帧
电气特性	理解欠缺	GPIO 驱动能力、上拉/下拉、电平兼容性影响代码逻辑

3. 安全合规无法保证

在汽车电子、机器人等领域，代码不仅要“能运行”，还必须满足功能安全和编码合规要求。

- MISRA-C包含大量规则，AI生成代码极易违反其中多条要求。
- ISO 26262强调软件可追溯性，而AI生成代码天然缺少从需求到实现的完整追溯链。

4. 实时性约束不可见

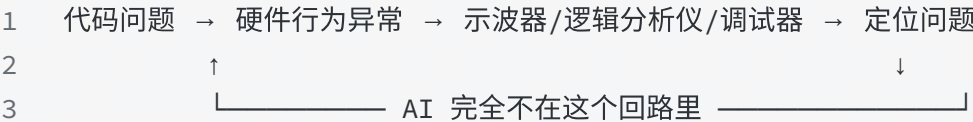
AI通常难以感知MCU运行时的真实资源和实时性限制，容易生成“逻辑正确但部署失效”的代码。

- 任务栈有限，局部大数组可能导致堆栈溢出。
- ISR中放置浮点运算或printf会直接破坏实时性。
- 若不理解RTOS优先级和资源占用关系，容易引入优先级反转和死锁问题。

5. 调试反馈回路断裂

嵌入式开发的问题定位依赖示波器、逻辑分析仪、JTAG调试器和硬件现场现象，AI天然不处在这个反馈回路中，难以像Web开发那样形成“生成—运行—报错—自动修复”的完整闭环。

代码块



6. 资源约束感知缺失

AI倾向生成“语义上正确但资源不经济”的代码，而嵌入式平台往往对RAM、Flash、调用层级和执行效率有严格约束。

- 使用malloc动态内存存在无MMU MCU上风险极高。
- 用浮点替代定点在无FPU芯片上代价巨大。
- 深层函数调用会额外消耗栈空间。
- 若不理解Flash/RAM分区限制，容易出现.text、.bss超限。

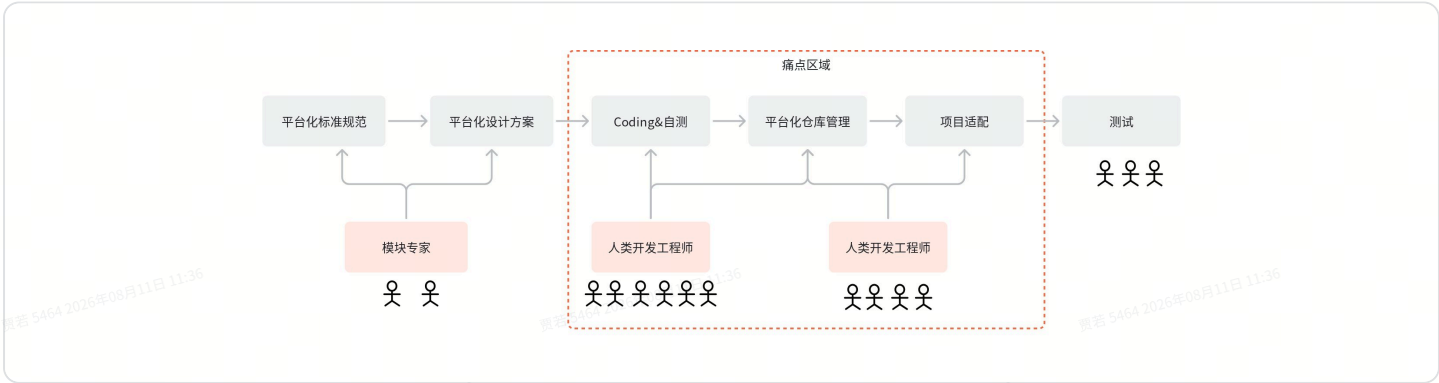
🎨 综上，面对配置代码手写带来的效率与质量瓶颈，不能简单寄希望于通用AI直接替代人工。在嵌入式驱动开发场景中，AI只有被放入**明确的规范边界、上下文供给、自动校验和反馈闭环**之中，才有可能稳定地产出可用结果。这也引出了CDD平台化2.0的核心方向：不是“直接上AI”，而是“**以工程化框架驾驭AI**”。

1.3 CDD平台化2.0规划

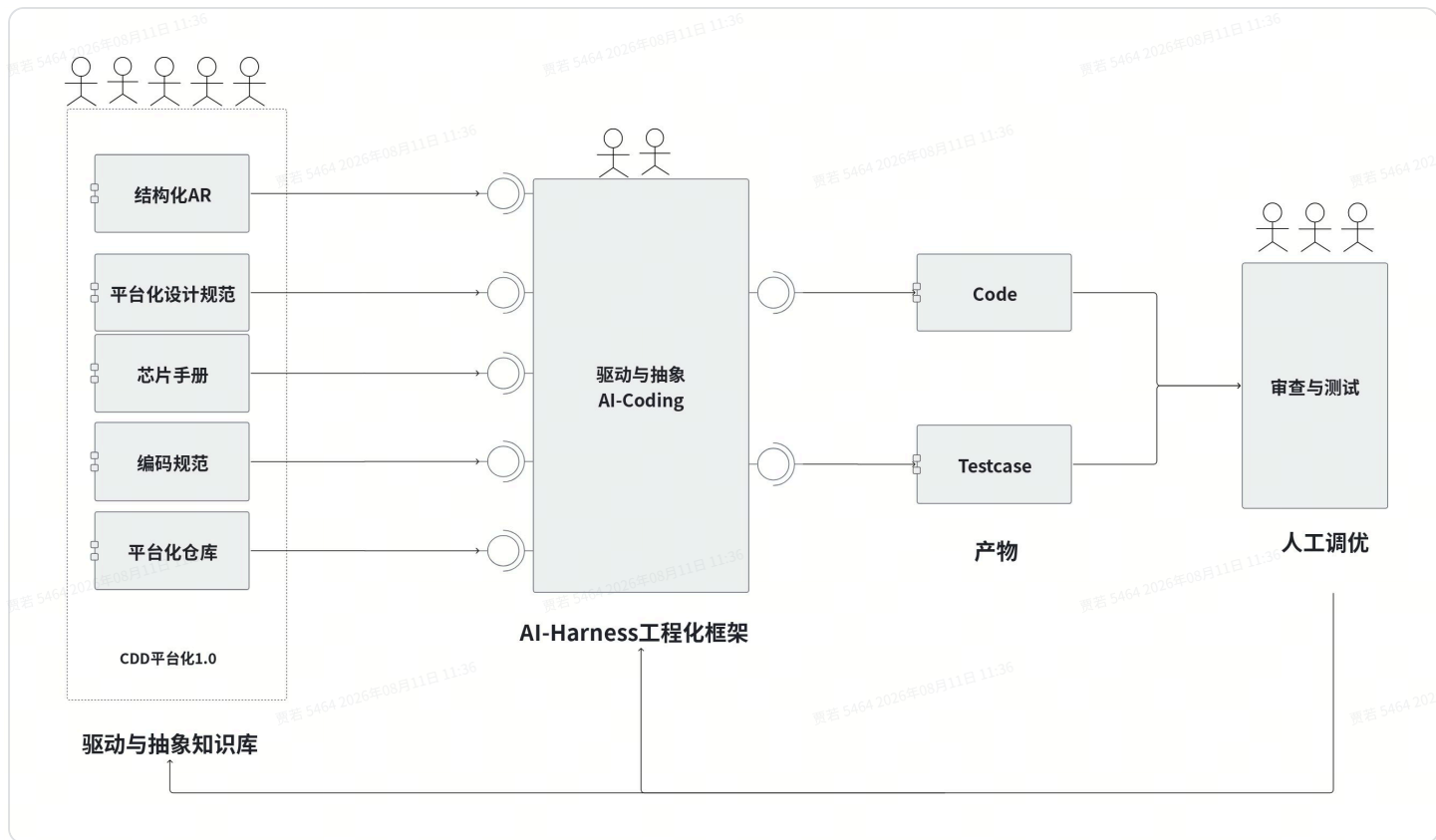
基于前述分析，CDD平台化2.0的目标并不是简单把“人工手写配置代码”替换为“AI自动写代码”，而是在CDD平台化1.0基础上，引入**Harness工程化框架**，对AI的知识输入、任务执行、结果校验和过程留痕进行系统化约束，使AI首先在配置代码这一结构化程度高、规则边界清晰、易于验证的场景中形成稳定闭环。

下图所示，CDD平台化2.0相较于1.0的变化，不在于替换原有平台化流程，而在于在流程之上叠加一层面向AI的工程化控制层，使配置代码生成从“依赖人工经验执行”转向“在规则约束下自动生成并自动校验”。

CDD1.0



CDD2.0



换言之，CDD平台化2.0的核心，不是追求一个“更会写代码”的模型，而是建设一套让AI在受控环境中持续输出正确结果的工程机制。该机制需要回答四个问题：

- **AI应当知道什么：**如何给到准确、充分但不过载的上下文；
- **AI不该做什么：**如何把项目规范、禁用项和架构边界转化为硬约束；
- **AI做完后如何判断对错：**如何通过自动化校验机制将结果纳入质量门禁；
- **AI如何持续改正：**如何通过闭环反馈让其围绕目标反复修正输出。

在这一思路下，CDD平台化2.0并不试图一步到位覆盖全部嵌入式开发任务，而是选择配置代码生成作为第一落点。原因在于，配置代码相较于驱动逻辑代码，具有输入边界更清晰、输出形式更标准、规则更显性、静态校验更容易实施等特点，更适合作为AI工程化的首个突破口。

因此，CDD平台化2.0可以概括为：



以CDD平台化1.0为基础，以配置代码生成场景为切入点，以Harness工程化框架为约束手段，构建可验证、可评测、可追溯的AI辅助开发闭环。

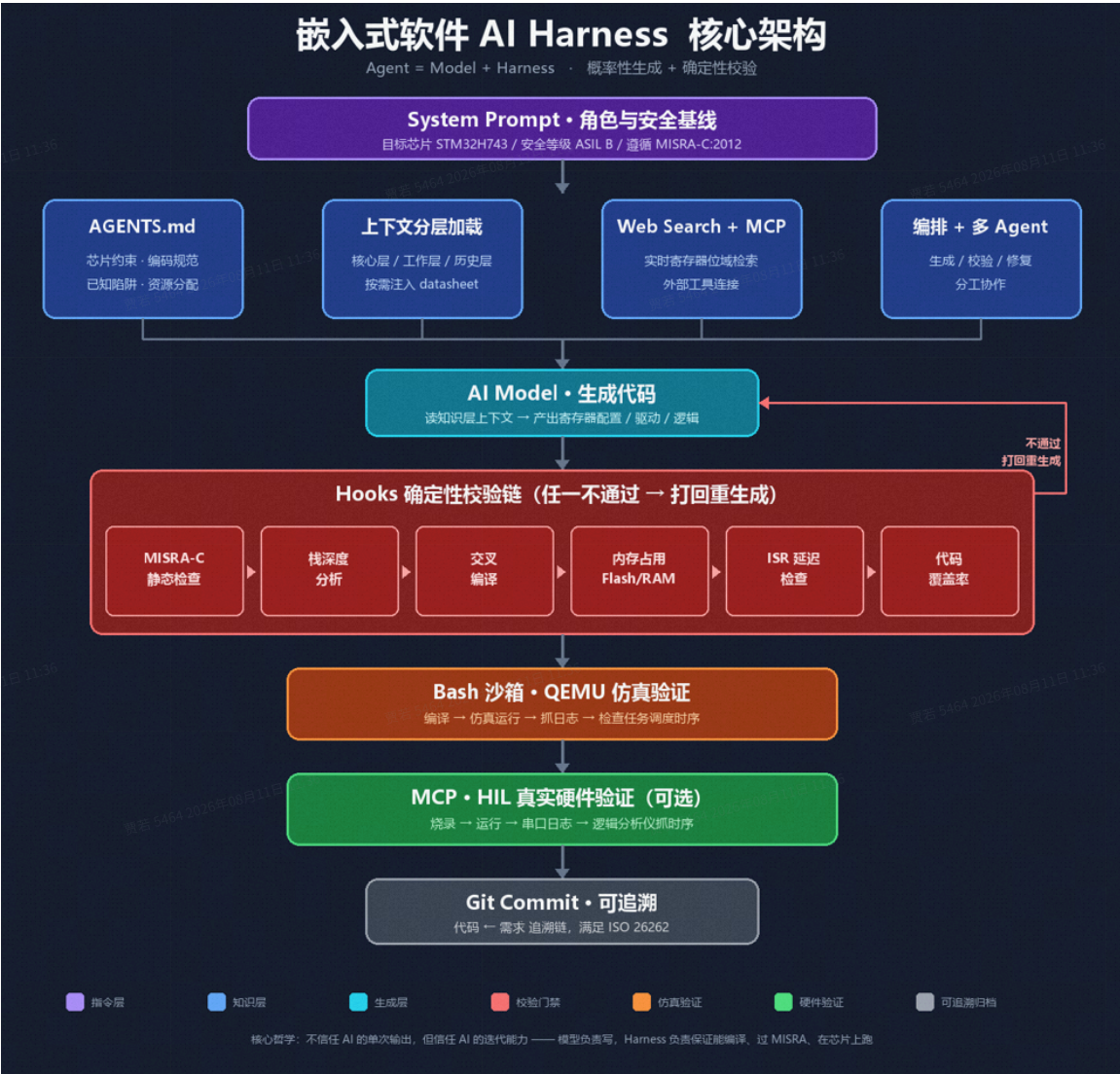
1.4 核心技术点说明

为支撑CDD平台化2.0目标，关键不在于单一模型能力提升，而在于建立一套完整的工程化支撑机制。该机制需要同时解决“知识如何提供”“规范如何承载”“结果如何校验”“任务如何编排”“执行如何受控”等问题。Harness Engineering正是当前能够将AI从“会生成代码”推进到“可在工程体系中稳定产出”的核心方法。

因此，以下技术点并非彼此独立的功能堆叠，而是共同服务于同一目标：在CDD平台化1.0基础上，构建面向配置代码生成的AI工程化闭环。

其中，AGENTS.md负责承载项目规则入口，上下文工程负责精准供给知识，Hooks负责建立质量门禁，MCP负责连接外部资源与工具，编排与文件系统负责组织任务过程，沙箱负责提供受控执行环境。各技术点协同后，才能使AI输出满足可用、可控、可追溯的要求。

1.4.1 Harness工程化框架介绍



Harness 组件	通用作用	嵌入式映射	角
AGENTS.md 记忆	跨会话知识注入	芯片规格、编码规范、平台化仓库、已知缺陷	训练数据不足、
上下文工程	对抗上下文溢出	分层加载 datasheet / HAL / 需求	上下文多源
Hooks 校验	确定性拦截	MISRA-C、栈分析、编译、资源检查	安全合规难以保
Bash + 沙箱	自我验证循环	交叉编译 + Renode/QEMU 仿真	实时性约束不可
Web Search + MCP	实时知识 + 外部工具	datasheet 检索、OpenOCD、逻辑分析仪	训练数据不足、
编排 + 文件系统	多 Agent 协同 + 试错回滚	分工协作 + Git 版本追溯	全局

Harness Engineering（驭驭工程）是2026年初由OpenAI和Anthropic同时推动的AI工程化范式。Harness Engineering的核心思想不是增强模型“自由发挥”的能力，而是通过环境、规则、反馈和执行机制去约束模型产出，使AI从“偶尔写对”变成“在规定边界内持续做对”。

Harness Engineering的有效性已有充分数据验证。LangChain 2026年3月发布《The Anatomy of an Agent Harness》实验报告：同一底层模型，仅更换Harness架构（模型权重不变），Terminal Bench 2.0通过率从52.8%提升至66.5%，提升13.7个百分点。OpenAI Codex实践数据显示，约70%的代码PR可由Agent独立完成并一次通过，仅10%需要人工介入；一个不到10人的团队在5个月内通过Harness系统交付了超过100万行生产代码，全程无人工编写。

💡 对于本项目而言，Harness的意义在于：它能够把CDD平台化仓库中的规范、模板、质量要求和历史经验，转化为**AI可消费、可执行、可校验**的工程约束，从而支撑配置代码生成场景稳定落地。

1.4.1.1 Agents.md记忆

AGENTS.md是放置在代码仓库根目录的结构化Markdown文件，用于向AI Agent声明项目的技术栈、禁止API、编码规范、架构约束等。它不是给人看的README，而是给Agent的“系统提示词”——随代码演化、被版本控制、被团队共同维护。

核心设计原则——渐进式披露（Progressive Disclosure）：

- AGENTS.md本身不存储具体知识，只告诉Agent“去哪里找”
- 约100行，只做目录索引，指向docs/architecture/、docs/conventions/等子文档
- Agent按需加载对应文档，避免一次性塞入几百页文档导致上下文溢出

在本项目中，AGENTS.md将作为CDD平台化仓库面向AI的统一入口，承载配置代码生成相关的目录索引、设计规范、编码约束、禁用项和参考文档路径。其作用不是存储所有知识，而是引导Agent按需获取正确信息，降低上下文混乱和错误引用的风险。

1.4.1.2 上下文工程

上下文工程解决“Agent不知道”的问题，核心是对抗信息腐烂和上下文溢出。

核心机制：

1. 上下文重置机制（Context Reset Architecture）

- 每3-5个回合强制清除大模型对话历史
- 清除前将AI生成的代码、未解决的Bug列表序列化存入本地进度文件（如claude-progress.txt）
- 新会话从轻量文件恢复，带着“干净的大脑”继续干活

2. 动态上下文加载

- 根据任务类型动态加载对应文档，而非一次性塞入全部上下文
- 通过Git版本控制确保上下文文档一致性

3. 上下文压缩与清洗

- 动态筛选、压缩、清洗上下文信息
- 降低token消耗，提升响应速度

关键洞察：上下文不是越多越好，而是越精准越好。

在本项目中，上下文工程将重点围绕芯片手册、平台化设计规范、配置模板、历史案例和缺陷经验进行按需加载，使AI在生成配置代码时能够基于正确、精简、与任务强相关的信息工作，从而降低幻觉和上下文污染问题。

1.4.1.3 Hooks校验

Hooks校验解决"Agent不该做"的问题——通过机械化的静态验证手段，将架构约束从"建议"变为"强制"。

约束强度金字塔（从上到下可靠性递减）：

1. **CI/CD门禁拒绝合并（最强）：** 代码不通过则无法入库
2. **Pre-commit钩子：** 本地提交前拦截违规代码
3. **Linter规则：** 静态代码分析自动检测
4. **AGENTS.md自然语言约束（最弱）：** AI可能选择性忽略

在本项目中，Hooks校验将重点用于约束配置代码生成结果满足QAC、编码规范、平台化接口要求及架构边界，并通过“违规检测—错误反馈—自动修复”的方式，将自然语言规范转化为真正可执行的质量门禁。

1.4.1.4 Web Search+MCP

MCP（Model Context Protocol，模型上下文协议）是Anthropic于2024年11月发布的开放标准协议，被称为"AI领域的USB-C接口"。

核心架构（Client-Server模式）：

- **MCP Host：** 运行环境与入口（如Claude Desktop、Cursor IDE）
- **MCP Client：** 内嵌于Host中，与Server建立连接
- **MCP Server：** 由数据/工具所有者部署，掌控自身资源和认证

与AGENTS.md的关系： 互补而非竞争。MCP是运行时协议（Agent如何动态交互工具），AGENTS.md是静态上下文（Agent在项目里应遵循的规范）。类比：MCP是"操作系统API"，AGENTS.md是"项目README for machines"。

在本项目中，MCP的主要价值不在于追求工具数量，而在于为后续知识获取、代码搜索、配置生成、仿真验证和工具链接入预留标准化接口，使AI能力能够逐步接入更丰富的工程环境。

1.4.1.5 编排+文件系统

编排解决"Agent不知道怎么做事"的问题，核心是将复杂任务拆解、分配、协调。

阶梯式Agent职责分工（Multi-Agent Hierarchy）：

角色	职责	特点
Planner（规划者）	在本地TODO.md中拆解步骤	只规划不执行
Executor（执行者）	改特定文件	只执行不规划
Reviewer（审查者）	对照架构文档审查代码	使用更强推理模型

文件系统与版本管理：

- 标准化目录结构：AGENTS.md（入口）+ docs/（文档库）+ scripts/（自动化）+ harness/（状态管理）
- harness/memory/存储经验教训库，harness/trace/存储执行轨迹日志
- 支持一键回滚历史版本，保障迭代稳定性

编排循环（四步标准流程）：

1. 环境初始化（Setup）：初始化沙箱、加载配置、注入规则
2. 受控执行（Execution）：Agent在约束下执行任务
3. 结果校验与拦截（Check）：Hooks校验输出合规性
4. 资源回收与归档（Teardown）：释放资源、归档日志

在本项目中，编排机制将围绕“任务拆解—受控执行—自动校验—结果归档”的标准流程展开，文件系统则用于沉淀规则、记忆和执行轨迹，保证配置代码生成过程可复现、可追踪、可回溯。

1.4.1.6 Bash+沙箱

沙箱解决"Agent在哪里干活"的问题——为Agent提供隔离的执行环境，所有代码执行、工具调用、数据操作都在受控环境运行。

核心能力：

- 每个Git工作树对应独立沙箱，避免任务间状态污染
- 自动化反馈闭环：AI改代码后立即自动触发lint/test，错误堆栈自动转为Prompt喂回AI，AI在后台自己跟报错死磕10个回合，直到测试全部通过
- 安全隔离：避免AI误操作导致的系统风险

在本项目中，沙箱将作为AI执行配置生成、自检和修复的受控环境，避免任务间相互污染，并支撑“生成后立即触发检查、检查结果自动回灌、AI继续修复”的工程闭环。

2. 五看分析

2.1 看行业

从行业发展阶段看，AI编码已从早期的代码补全和问答辅助，逐步进入强调约束、验证和流程集成的工程化落地阶段。尤其是2026年以来，Harness Engineering、AGENTS.md、MCP等方法与标准快速成熟，说明行业关注点已经从“模型能否生成代码”转向“模型能否在工程体系中稳定交付结果”。

对于嵌入式和汽车电子软件开发而言，这一变化尤为关键。原因在于，该领域对代码正确性、可追溯性、资源约束和质量门禁的要求远高于通用软件开发，单纯依赖Prompt提升生成效果的方式难以满足工程落地需求。而Harness等工程化手段的出现，为AI进入强约束开发场景提供了现实路径。

同时，汽车电子软件价值占比持续提升，软件定义汽车趋势推动底层软件复杂度快速上升，传统依赖人力扩张的开发模式正逐步接近效率上限。在此背景下，AI辅助开发的价值已经从“锦上添花”转变为“提升复杂系统交付能力的必要探索方向”。

总体来看，行业已经具备推进AI工程化落地的外部条件。对于CDD平台化2.0而言，这意味着当前讨论的不再是一个超前概念，而是一个具备现实技术基础和业务牵引的落地方向。

2.2 看竞争

直接竞争（汽车电子驱动软件领域）：

对手	方向	特点
芯片厂商SDK	底层驱动库AI辅助生成	如NXP S32K5系列已标注AI辅助生成代码；上游“污染”
光庭信息	AUTOSAR Agent（配置层AI化）	从AUTOSAR配置切入，非CDD驱动层
东软睿驰	CAPI（代码级标准化）	从AUTOSAR AP架构标准化切入，侧重上层
翱韬智能	Altior Agent（物理硬件调试）	从电机控制切入，聚焦实物调试闭环

从竞争格局看，行业内围绕AI与嵌入式/汽车电子结合的探索已经展开，但不同方案的切入层次并不相同。已有方案大致集中在三类方向：一类从AUTOSAR配置等上层配置场景切入，一类从MATLAB/仿真/代码生成等工具链场景切入，另一类从示波器、调试器、电源等物理硬件联动场景切入。

这说明行业已经普遍认识到，嵌入式开发中的AI应用不能只依靠通用对话能力，而必须与具体工程环节结合。但从现有公开方案看，围绕CDD驱动平台化基础、以配置代码生成作为第一落点、并通过Harness工程化机制保障输出可验证和可追溯的路径，尚未形成直接同类竞争方案。

团队在这一方向上的潜在差异化优势在于：并不是从零开始打造一个泛化的嵌入式AI平台，而是依托已有CDD平台化1.0成果，在既有模块仓库、设计规范和质量体系上叠加AI工程化能力。这种路径更聚焦，也更容易形成短期可验证成果。

因此，从竞争角度看，CDD平台化2.0并非进入一个完全拥挤的红海领域，而是选择了一个具备现有基础、可形成差异化定位的细分切入口。

2.3 看自己

从团队自身条件看，推进CDD平台化2.0并非从0到1的全新探索，而是在既有平台化和质量体系基础上的进一步演进。当前最重要的内部前提，是CDD平台化1.0已经完成驱动逻辑与配置代码分离，这使得“配置代码生成”成为一个边界相对清晰、风险相对可控、收益相对直接的目标场景。

与此同时，团队在AI辅助开发、静态检查和规范体系方面也具备初步基础。例如，已有AI Skill和工作流实践可以为后续Agent化能力建设提供经验，QAC静态检查和平台化设计规范则为代码质量门禁和规则显性化提供了现成抓手。这些能力组合在一起，使团队具备将“人工经验”逐步转化为“工程约束”的现实条件。

当然，也应看到当前仍存在明显差距：团队虽有平台化基础，但尚未形成面向AI的系统化上下文组织方式、规则承载方式和自动反馈闭环；已有规范和检查能力更多服务于人工开发流程，尚未完全转化为可直接驱动AI执行的工程资产。因此，CDD平台化2.0本质上既是一次能力建设，也是一次知识结构化和流程再工程化的过程。

结合现有情况，团队基础与差距可归纳如下：

已有基础：

1. CDD平台化1.0已完成

- 驱动逻辑与配置代码成功分离
- 模块专家小组→软件开发→平台化仓库→项目配置的标准化流程已建立
- 平台化模块统一管理，多项目复用

2. AI辅助开发已有初步实践

- AI Skill实现多模块代码100% AI检查
- AI工作流落地模块案例

3. 质量体系成熟

- QAC静态检查成熟
- 平台化驱动设计规范比较完备

当前差距：

维度	现状	目标
配置代码生成	人工手写	AI工程化生成
AI代码质量保障	人工审查+AI Skill检查	Harness框架自动化校验
架构约束	自然语言规范	import-linter/QAC门禁强制
反馈闭环	人工运行测试→人工反馈	自动触发→自动喂回→自动修复
知识管理	散落在个人经验	AGENTS.md结构化声明+版本控制



总体而言，团队已经具备推进CDD平台化2.0的起步条件，但成功关键不在于单纯引入模型，而在于能否把已有平台化成果、规范体系和质量门禁转化为AI可消费、可执行、可校验的工

2.4 看机会

综合行业趋势、竞争格局和团队基础，当前推进CDD平台化2.0具备较好的窗口期。**一方面**，Harness Engineering、AGENTS.md、MCP等关键方法和标准已趋于成熟，外部生态已经提供了可直接借鉴和复用的基础设施；**另一方面**，团队内部正面临多项目并行、人力瓶颈显现、交付节奏加快的现实压力，迫切需要寻找一种既能提升效率、又不显著放大质量风险的新方式。

更重要的是，CDD配置代码场景本身具备良好的试点条件：其结构化程度高、规则边界清晰、与现有平台化仓库高度耦合，便于快速建立评测机制和成效闭环。这意味着项目既可以避免一开始就挑战最困难的驱动逻辑自动生成问题，又能在较短周期内验证AI工程化方案的实际价值。

因此，当前的机会并不只是“AI很热”，而是外部方法成熟、内部需求迫切、试点场景合适三者同时具备。对于CDD平台化2.0而言，这种窗口期具有较强的时效性：越早启动，越有机会在团队内部建立方法先发优势和工程资产积累。

基于上述判断，当前机会主要体现在技术与业务两个层面：

技术机会：

1. Harness Engineering正处于爆发期

- 2026年初OpenAI Codex实验引爆行业关注
- 行业共识"Agent = Model + Harness"已形成
- 开源生态（LangGraph 0.3.x）已成熟，可基于其构建
- 时间窗口：当前是早期入场期，1-2年内建立先发优势

2. AGENTS.md + MCP标准已成熟

- AGENTS.md：55+工具支持、41,000+项目采用，事实标准已确立
- MCP：1000+开源服务器，Anthropic/OpenAI/Google三巨头支持
- 可直接复用，无需自建标准

3. 汽车电子AI化窗口期

- 芯片厂商SDK已开始AI辅助生成，但CDD驱动层尚无人系统化布局
- AUTOSAR CAPI从架构标准化迈向代码标准化，CDD驱动层可同步跟进

业务机会：

1. 多项目并行交付效率提升

- ACU/BCM-D/CCU/HWA/BZCU/PZCU等多项目并行，人力瓶颈明显
- AI工程化可提升配置代码生成效率，缓解人力瓶颈



综上，当前从配置代码场景切入CDD平台化2.0，具备“方向可行、节奏合适、收益明确”的综合条件，是一个风险相对可控、价值相对清晰的切入时机。

3. 技术分析

3.1 技术点分析

在完成问题识别和可行性判断后，第3章需要进一步回答一个关键问题：CDD平台化2.0在技术上应当从哪里切入，才能既控制风险，又尽快形成有效成果。

从整体目标看，AI在嵌入式驱动软件中的应用空间很大，但不同任务的技术成熟度、规则显性程度、验证难度和工程风险差异明显。如果不进行场景分层，而是一开始就尝试让AI直接承担复杂驱动逻辑开发，不仅对模型能力要求过高，也会显著放大上下文、合规性和验证闭环等问题。

结合CDD平台化1.0现有基础，当前最合理的技术路径，不是从“最复杂的代码”切入，而是**从结构化程度最高、规则最明确、最容易自动校验的代码类型切入**。配置代码恰好具备这一特征：其输入通常来自芯片资源、项目需求、引脚映射、模块参数等相对明确的信息；其输出通常为宏定义、结构体初始化、枚举映射、表项配置等标准化形式；其正确性也更容易通过静态规则、脚本检查和编译验证进行快速判断。

从这个角度看，CDD平台化1.0已经为CDD平台化2.0创造了非常关键的前提条件。由于驱动逻辑与配置代码已经分离，AI不需要在一开始就理解完整的底层驱动实现机制，而可以先聚焦于配置层的生成与修正任务。这种“先收敛场景、再逐步扩展”的路径，更符合嵌入式AI工程化的实际规律。

因此，技术实施路径应明确分阶段推进：

- **Phase 1：先从配置代码AI生成突破**

重点解决配置项输入、配置代码生成、静态规则校验、错误回灌修复和过程留痕问题，建立基础闭环。

- **Phase 2：再扩展到驱动逻辑代码AI辅助**

在配置层能力稳定后，再逐步引入芯片手册、平台驱动框架和模块交互关系等复杂上下文，支持驱动逻辑辅助开发。

- **Phase 3：最终实现全流程AI工程化**

在局部场景成功基础上，再向需求理解、设计生成、测试构建、缺陷修复等更完整流程扩展。



这一技术路径的核心思想是：不是让AI直接挑战最难的问题，而是先在高确定性场景中建立“可控生成—自动校验—持续修复”的工程能力，再逐步向高复杂度任务迁移。

3.2 核心技术价值分析

从表面上看，CDD平台化2.0是在现有平台化体系中引入AI能力；但从技术本质上看，其真正价值并不在于“多了一个代码生成工具”，而在于把AI输出纳入工程体系管理，使其具备可约束、可验证、可追溯、可积累的属性。这也是CDD平台化2.0与一般意义上“用大模型写代码”的根本区别。

具体来看，CDD平台化2.0的核心技术价值主要体现在以下几个层面：

技术价值点	说明
将自然语言规范转化为工程化约束	传统开发中，大量规范依赖资深工程师理解和执行，存在认知偏差和落地不通过AGENTS.md、Linter、QAC、Hooks等机制，可以把“建议性规则”变为“可执行门禁”，使规范真正进入开发闭环。
将分散知识转化为可供AI消费的上下文资产	芯片手册、平台设计规范、历史案例、缺陷经验、模块模板等知识以往分散在代码和个人经验中。通过上下文工程进行结构化组织后，这些知识能够被AI调用，从而降低对通用训练数据的依赖。
将一次性生成能力转化为可持续复用的生产能力	如果没有工程化闭环，AI每次生成结果都可能高度依赖Prompt和个人操作。通过标准化编排、自动校验和反馈修复机制，可以使生成过程逐步固化，形成可复用的任务流。
将历史错误转化为组织级经验资产	每一次错误修复、规则补充、禁用项沉淀，本质上都在完善团队自己的知识库。CDD平台化2.0不仅输出代码，更持续沉淀一套面向嵌入式驱动场景的经验资产。
将局部试点能力转化为后续扩展基础	配置代码场景的成功，不仅能直接提升当期交付效率，更重要的是能够验证嵌入式场景中的有效性，为未来扩展至驱动逻辑、测试构建乃至更完整流程奠定基础。

因此，CDD平台化2.0的核心技术价值，并不是“AI把人工代码写快一点”，而是建立一套适用于CDD驱动层的AI工程化方法，使AI输出从“可生成”走向“可落地、可治理、可积累”。

4. 价值说明

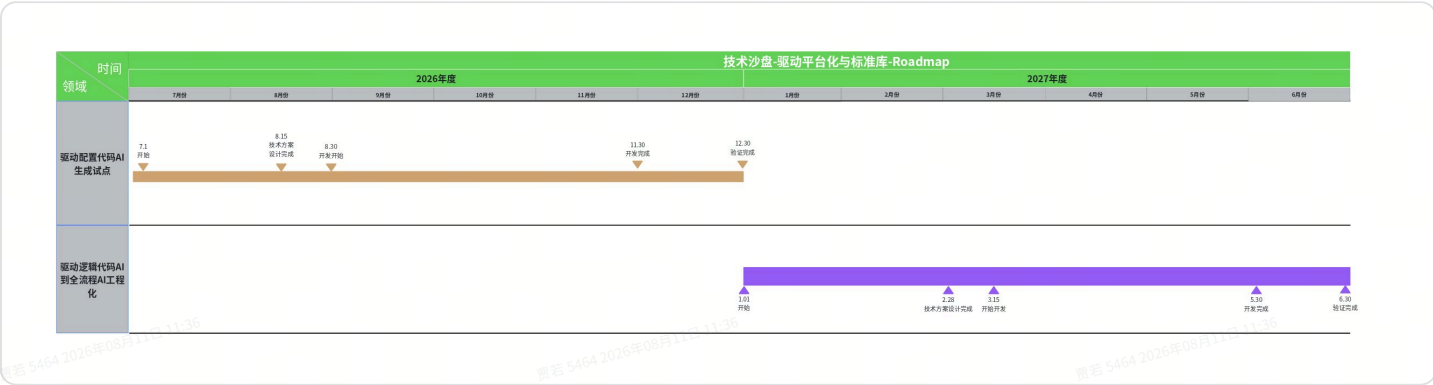
4.1 技术价值

- 建立CDD驱动层AI工程化范式：**当前行业AI+嵌入式方案集中在配置层（光庭AUTOSAR Agent）、工具链层（MathWorks MCP）、物理调试层（翱韬Altior Agent），从CDD驱动平台化+Harness工程化框架组合切入尚属空白，可建立技术先发优势
- 将自然语言规范变为机械化约束：**当前开发痛点“风格难以统一”“手写易出错”的根本原因是规范靠人执行。Harness通过import-linter/QAC门禁将规范从“建议”变为“强制”，从机制上消除人为偏差
- 构建可积累的AI工程资产：**AGENTS.md中每一条规则都对应一次历史失败，随着使用积累，护栏库持续完善，形成团队独有的、可版本控制的、可传承的工程知识资产
- 突破嵌入式AI数据瓶颈：**嵌入式C训练数据稀缺是行业共性痛点。通过CDD平台化仓库的结构化代码+芯片手册上下文工程，可在不依赖通用训练数据的情况下，使AI在受限领域内可靠工作

4.2 业务价值

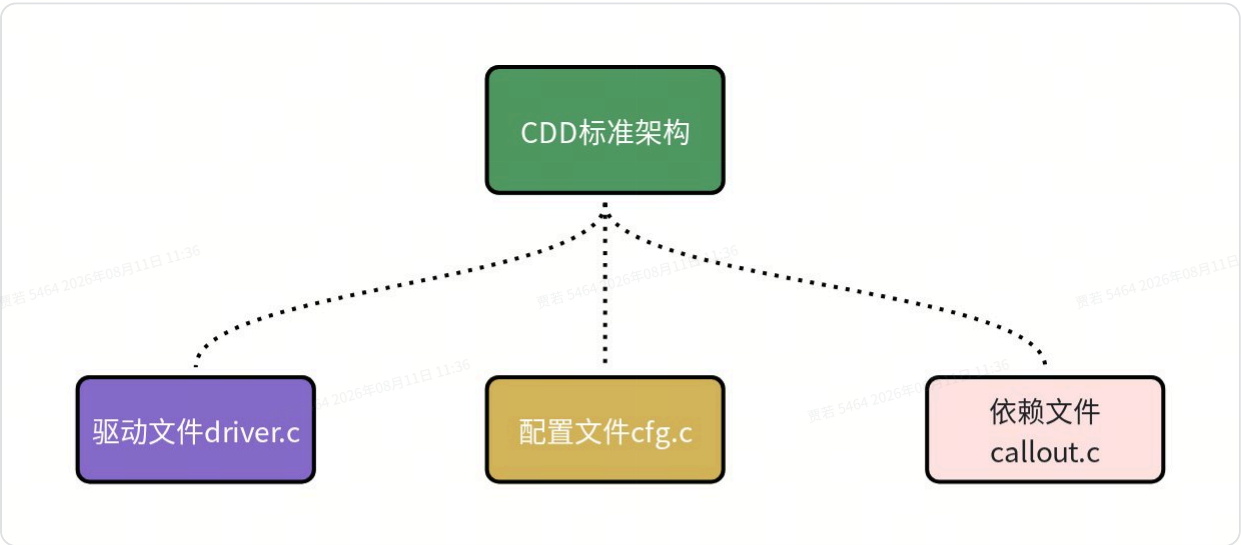
- 1. 多项目并行交付效率提升：当前ACU/BCM-D/CCU/HWA/BZCU/PZCU等多项目并行，人力瓶颈明显。配置代码AI生成可提升交付效率，支撑更多项目并行，预估提效2~3倍
- 2. 量产质量指标持续达标：FFR<0.025%、千车工单率<0.054%、无安抛——AI生成+Hooks自动校验可进一步压缩人为错误，保障质量指标
- 3. 国产化芯片导入加速：19篇国产芯片导入规范已有基础，AI辅助可降低导入门槛、加速国产替代进程
- 4. 降低人员依赖与知识传承成本：当前嵌入式开发知识高度依赖资深工程师个人经验。AGENTS.md将隐性知识显性化，降低新人上手和跨模块协作门槛

5. 技术路标 (Road map)



Backup

CDD标准架构





harness_engineering_guide.pdf

31.58MB



https://mp.weixin.qq.com/s/GkhemHUAhKWV-3Uxaa1Mqg?scene=1&click_id=1191804863